

INSTITUTE OF ADVANCED TECHNOLOGICAL SCIENCES

Undergraduate Curriculum & Course Syllabi

Bachelor of Science in Computer Science & Systems
Engineering • Rigorous STEM Analytics Specification.

Department: Department of Computer Science and Engineering

Academic Year: 2026 – 2027 Framework

Accreditation Statement: Confirms full compliance with international engineering core frameworks and mathematical evaluation protocols.

Program Structure & Degree Requirements

The Bachelor of Science in Computer Science and Systems Engineering degree scheme requires the successful completion of a minimum of 120 semester credit units. This curriculum provides deep foundational knowledge in abstract computer systems engineering, mathematical evaluation parameters, dynamic optimization strategies, and relational database engine theory.

Credit Distribution Profile

Category Code	Curriculum Core Category Target	Required Semesters	Credit Units
CS-CORE	Core Computer Science & Software Design Sequences	6 Semesters	48 Units
MATH-STEM	Advanced Engineering Mathematics, Calculus, & Formal Logic	4 Semesters	24 Units
SYS-ENG	Systems Architecture, Operating Environments, & Concurrency	3 Semesters	18 Units
DATA-REL	Relational Calculuses, Relational Engine Design, & Information Storage	2 Semesters	12 Units
ELECT-ADV	Advanced Technical Specializations & Project Capstones	2 Semesters	18 Units
TOTAL	Minimum Required for Degree Validation	8 Semesters	120 Units

Academic Progression Invariants

Students must maintain a cumulative GPA of 3.0 out of 4.0 within all primary technical domains. Standard computational modules carry explicit dependency constraints. No student is permitted to enroll in advanced systems design sequences or dynamic data structures classes without proving master-level competence in basic procedural abstraction structures and formal math tracks.

Curriculum Mapping & Sequence Grid

The matrix below traces the recommended course scheduling profile across the standard eight-semester enrollment path. This sequence prevents pipeline blocks and ensures appropriate preparation for advanced coursework.

Complete Curriculum Matrix

Semester	Course Code	Course Descriptive Title	Credit Units
Semester I	CS-101	Introduction to Algorithms & Abstraction Structures	4.0
	MATH-111	Mathematical Calculus & Analytical Series Sequences	4.0
Semester II	CS-102	Advanced Object-Oriented Framework Designs	4.0
	MATH-112	Discrete Algebraic Networks & Combinatorics	4.0
Semester III	CS-201	Data Structures & Non-Linear Structural Analysis	4.0
	SYS-211	Computer Architecture Layouts & Assembly Primitives	3.0
Semester IV	CS-202	Design and Mathematical Optimization of Algorithms	4.0
	DATA-221	Relational Database Theory & Storage Architecture	3.0
Semester V	SYS-311	Operating Systems Engineering & Thread Parallelisms	4.0
	DATA-321	Advanced Transaction Engines & High-Load Tuning	3.0
Semester VI	CS-302	Advanced Python Environments & Runtime Mechanics	4.0
	SYS-322	Distributed Networks & Cooperative Async Processes	3.0
Semester VII	ELECT-401	Advanced Specialization: Metaprogramming Systems	3.0
	CAP-491	Senior Design Project Capstone — Phase I	3.0
Semester VIII	ELECT-402	Advanced Specialization: Distributed Ledger Graphs	3.0
	CAP-492	Senior Design Project Capstone — Phase II	6.0

Course Syllabus: CS-102 Advanced Object-Oriented Framework Designs

CS-102 Advanced Object-Oriented Framework Designs

Credits: 4.0 Semester Units

Contact Hours: 45 Lecture, 30 Lab

Prerequisites: CS-101

Course Catalog Description

Provides an in-depth study of object-oriented program development using Python. The course covers structural memory layouts, pointer reference tracking, multiple inheritance graph optimization, method resolution behaviors, runtime attribute descriptors, and dynamic structural encapsulation interfaces.

Instructional Units & Content Breakdown

Unit 1: Low-Level Runtime Topologies and Memory Architecture

9 Hours

Analysis of the standard CPython runtime abstraction layer. Structural composition of the base item descriptor structure (*PyObject*). Tracking allocation metrics via real-time reference counting routines. Evaluating the limitations of reference counting when processing cyclic pointer graphs. Generational garbage collection topologies (*G_0*, *G_1*, *G_2*) and multi-tier heap traversal thresholds. Object address allocation optimization via literal string and integer interning spaces.

Unit 2: Multiple Inheritance Protocols and Graph Resolution

9 Hours

Designing complex multi-parent inheritance hierarchies. Abstracting inheritance dependencies as Directed Acyclic Graphs (DAGs). Resolving tracking hierarchies using the C3 Linearization algorithm. Formal mathematical merge rules for sequence linearization. Monotonicity constraints and handling local precedence order violations. Preventing compilation errors caused by non-linearizable tracking graphs.

Unit 3: Runtime Interception, Metaclasses, and Descriptors

9 Hours

The class initialization lifecycle. Metaclasses as class constructors. Intercepting object allocation steps using custom structural metaclasses. Enforcing architectural rules, validating method signatures, and injecting telemetry code into objects at compilation. Deep analysis of the attribute descriptor protocol. Comparing data descriptors against non-data descriptors. High-priority resolution routing through the instance dictionary (`__dict__`) and fallback methods.

Required Textbooks & Academic Literature

- **Primary:** Martelli, A., Ravenscroft, A., & Ascher, D. (2023). *Python in a Nutshell: A Desktop Quick Reference* (4th ed.). O'Reilly Media.

- **Secondary:** Beazley, D. (2022). *Python Distilled* (1st ed.). Addison-Wesley Professional.

Course Syllabus: CS-201 Data Structures & Non-Linear Analysis

CS-201 Data Structures & Non-Linear Structural Analysis

Credits: 4.0 Semester Units

Contact Hours: 45 Lecture, 30 Lab

Prerequisites: CS-102, MATH-112

Course Catalog Description

Covers advanced data structures and their use in building efficient applications. Includes mathematical analysis of complexities, self-balancing tree structures, advanced array expansion techniques, interval slicing protocols, and custom high-efficiency mapping tracking systems.

Instructional Units & Content Breakdown

Unit 1: Formal Complexity Estimation & Amortized Cost Analysis

9 Hours

Rigorous mathematical definitions of Big-O, Big-Omega, and Big-Theta notation boundaries. Analyzing resource consumption inside computer systems. Introduction to amortized sequence analysis. The Accounting Method and the Potential Function Method ($\Phi(D)$) for proving long-term operational efficiency. Application of potential functions to track dynamic array resizing steps, demonstrating constant average-case insertions.

Unit 2: Custom Sequence Protocols & Index Slicing Topologies

9 Hours

Designing highly optimized sequence buffers. Supporting multidimensional lookup patterns and negative offset index conversions. Structural sequence slicing mechanics. Building index mapping pipelines that run in predictable time bounds. Managing internal data layouts to prevent memory fragmentation and ensure quick retrieval times.

Unit 3: Custom Hashing Systems & Collision Resolution Tracks

9 Hours

Theoretical analysis of custom hashing operations. Mapping keys to index buckets using hash formulas. Comparing open addressing mechanisms against separate chaining using linked nodes. Load factor thresholds ($\alpha = n/m$) and monitoring entry density. Designing strategies for dynamically resizing and re-hashing storage buckets to prevent runtime lookups from degrading toward linear performance.

Required Textbooks & Academic Literature

- **Primary:** Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- **Secondary:** Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. John Wiley & Sons.

Course Syllabus: CS-202 Design & Mathematical Optimization of Algorithms

CS-202 Design & Mathematical Optimization of Algorithms

Credits: 4.0 Semester Units

Contact Hours: 45 Lecture, 30 Lab

Prerequisites: CS-201

Course Catalog Description

Advanced strategies for designing and optimizing complex algorithms. The course focuses on non-linear interval tracking structures, graph pathfinding optimizations, network isolation components, and mathematical space optimization methods.

Instructional Units & Content Breakdown

Unit 1: Non-Linear Range Queries and Interval Trees

9 Hours

Building and navigating Segment Trees for efficient range queries. Dividing data sets into logarithmic interval components. Implementing lazy propagation techniques to defer sub-segment updates until explicitly requested. Maintaining interval query performance balances during frequent real-time updates.

Unit 2: Graph Path Optimization and Cycles

10 Hours

Advanced pathfinding on complex graph networks. The Bellman-Ford path optimization method and processing negative edge weights. Designing algorithms to detect negative weight cycles to prevent infinite path loops. Exploring Tarjan's Strongly Connected Components (SCC) algorithm. Tracking node connections using discovery timestamps and low-link values in a single Depth-First Search (DFS) pass.

Unit 3: State Vector Optimizations & Matrix Computations

8 Hours

Optimizing memory layout patterns for overlapping subproblems. Moving from linear data tracking arrays to compact state representations. Formulating sequence transitions as matrix multiplications. Using binary matrix exponentiation to find sequence values in logarithmic time and constant space, avoiding large storage structures.

Required Textbooks & Academic Literature

- **Primary:** Kleinberg, J., & Tardos, É. (2013). *Algorithm Design*. Pearson.
- **Secondary:** Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Course Syllabus: DATA-221 Relational Database Theory & Storage Architecture

DATA-221 Relational Database Theory & Storage Architecture

Credits: 3.0 Semester Units

Contact Hours: 45 Lecture

Prerequisites: CS-102

Course Catalog Description

Theoretical foundations of relational database management systems. Topics include functional dependencies, normalization algebra, schema design constraints, query compilation pathways, and physical index structures.

Instructional Units & Content Breakdown

Unit 1: Functional Dependencies & Closure Space Calculus

9 Hours

Relational model constraints and definitions. Formal analysis of functional dependencies ($X \twoheadrightarrow Y$). Deriving dependency closures using Armstrong's Axioms (Reflexivity, Augmentation, Transitivity). Computing minimal covers and superkey properties for database relations.

Unit 2: Database Schema Normalization Theory

10 Hours

Designing database schemas to eliminate anomalies. Step-by-step normalization from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF). Mathematical proofs for lossless-join decompositions and dependency preservation. Analyzing anomalies caused by improper schema design.

Unit 3: Query Optimization Paths & Index Lookups

8 Hours

How database engines execute SQL statements. Physical data layouts on disk. Comparing B+ Tree index structures and hashing indexes. Query execution paths: index seeks versus full table scans. Writing optimized queries that allow the engine to leverage index structures efficiently.

Required Textbooks & Academic Literature

- **Primary:** Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
- **Secondary:** Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

Course Syllabus: SYS-311 Operating Systems Engineering & Thread Parallelisms

SYS-311 Operating Systems Engineering & Thread Parallelisms

Credits: 4.0 Semester Units

Contact Hours: 45 Lecture, 30 Lab

Prerequisites: SYS-211, CS-201

Course Catalog Description

Explores the internal structure of operating systems, focusing on process management, concurrent task scheduling, synchronization mechanics, resource protection paths, and data streaming layouts.

Instructional Units & Content Breakdown

Unit 1: Concurrency Constraints and Thread Bottlenecks

9 Hours

Process management and thread scheduling models. Thread execution limitations within modern language engines. The CPython Global Interpreter Lock (GIL) and its impact on parallel performance. Differentiating between CPU-bound and I/O-bound bottlenecks. Strategies for bypassing interpreter locks using isolated process tasks.

Unit 2: Asynchronous Event Loops & Cooperative Schedules

9 Hours

Cooperative multitasking models. The structure of single-threaded asynchronous event loops. Coroutine suspension paths and non-blocking I/O operations. Task scheduling, callback routing, and managing concurrent states without multi-threading overhead.

Unit 3: Memory-Mapped Files & Efficient Data Ingestion

9 Hours

Virtual memory management and access strategies. Memory-mapped files (mmap) for fast data access. Bypassing user-space buffers to minimize application heap overhead during large file transfers. Processing stream data in fixed chunk sizes without duplicating object data in memory.

Required Textbooks & Academic Literature

- **Primary:** Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
- **Secondary:** Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.

Course Syllabus: DATA-321 Advanced Transaction Engines & High-Load Tuning

DATA-321 Advanced Transaction Engines & High-Load Tuning

Credits: 3.0 Semester Units

Contact Hours: 45 Lecture

Prerequisites: DATA-221, SYS-311

Course Catalog Description

Advanced transaction management and performance tuning in database management systems. Topics include concurrency anomalies, serializability analysis, locking protocols, and complex analytics using window functions.

Instructional Units & Content Breakdown

Unit 1: Transaction Concurrency Control & Serializability

9 Hours

Formal definition of ACID transactions. Analyzing concurrent execution schedules for conflict serializability. Building directed precedence graphs to detect conflict cycles. The Two-Phase Locking (2PL) protocol: Growing and Shrinking phase dynamics. Ensuring conflict-free schedules while preventing system deadlocks.

Unit 2: SQL Isolation Anomalies & Isolation Levels

9 Hours

The SQL standard isolation model. Detailed analysis of transaction anomalies: Dirty Reads, Non-Repeatable Reads, and Phantom Reads. Mapping isolation behaviors across different processing levels (Read Uncommitted up to Serializable). Performance trade-offs between strict data isolation and application throughput.

Unit 3: Complex Analytics and SQL Window Functions

9 Hours

Advanced SQL query design for high-load analytical tasks. Writing window functions using data partitioning syntax. Computing rankings, running totals, and moving averages across data subsets without collapsing rows. Using historical offset functions to identify patterns across sequential records.

Required Textbooks & Academic Literature

- **Primary:** Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
- **Secondary:** Celko, J. (2014). *Joe Celko's SQL for Smarties: Advanced SQL Programming* (5th ed.). Morgan Kaufmann.

Elective Track: Advanced Software Architecture Engineering

ELECT-401 Advanced Metaprogramming Systems

Credits: 3.0

Contact Hours: 45

Prerequisites: CS-302

Unit 1: Dynamic Compilation Hooks

9 Hours

Exploring compilation mechanisms in dynamic execution setups. Intercepting runtime imports, rewriting bytecodes, and modifying execution paths dynamically to inject diagnostic logging or security rules.

Unit 2: Dynamic Proxies and Object Wrappers

9 Hours

Designing proxy interfaces that control access to underlying components. Intercepting attribute updates, redirecting method calls, and implementing transparent caching layers without modifying the original class code.

Unit 3: Domain-Specific Languages (DSLs)

9 Hours

Building customized text processing tools within a host application language. Using parsing routines, handling tokens, and mapping text expressions to executable application logic.

Elective Track: Advanced Distributed Systems Architecture

ELECT-402 Distributed Ledger Graphs & Consensus Systems

Credits: 3.0

Contact Hours: 45

Prerequisites: SYS-322

Unit 1: Decentralized State Log Models

9 Hours

Mathematical foundations of immutable distributed state logs. Structuring state links using hash connections, validating blocks, and syncing state information across multiple network nodes safely.

Unit 2: Consensus Fault Tolerance Layouts

9 Hours

Analyzing consensus protocols within distributed networks. Reviewing Paxos, Raft, and Byzantine Fault Tolerant models. Managing network drops, processing delayed messages, and reaching a single state consensus safely.

Unit 3: Smart Contract Program Semantics

9 Hours

Designing secure programs for immutable distributed environments. Managing operational execution steps, sandboxing application logic, and verifying runtime resource constraints.

Laboratory Training Manual: System Intersections & Validation

The laboratory sequences translate theoretical engineering rules into practical system implementations. Students build functional prototypes that meet strict resource and correctness goals.

Laboratory Session I: CPython Memory Leak Detection & Cycle Resolution

Students intentionally build complex circular data structures to monitor memory usage patterns under variable garbage collection intervals. Tools are used to extract memory addresses and confirm the complete removal of cyclic data structures.

Laboratory Session II: C3 Linearization Trace Simulator

Building an execution tool that reads class layout diagrams and outputs the exact method execution order. The tool must identify non-linearizable patterns and raise appropriate compilation errors before starting execution.

Laboratory Training Manual (Cont.): Structural Analytics

Laboratory Session III: Dynamic Arrays potential Tracking Profile

Students build custom dynamic arrays that record real-time operational costs during heavy data insertion tests. The recorded costs are plotted against mathematical predictions to confirm the constant average-case runtime profile.

Laboratory Session IV: Segment Tree Propagation Tuning

Building a range monitoring system that updates data points across large datasets. Students run tests comparing standard updates against deferred propagation to evaluate performance differences across different range lengths.

Laboratory Training Manual (Cont.): Graph Operations

Laboratory Session V: Negative Cycle Path Interception

Building a routing prototype that handles mixed network edge weights. The system must find shortest paths and identify negative weight loops, outputting the exact circular path coordinates causing the instability.

Laboratory Session VI: SCC Mining in Microservice Topologies

Mapping network dependencies within a distributed microservices setup. Students use single-pass traversal routines to identify closely dependent service groups, helping optimize deployment layouts.

Laboratory Training Manual (Cont.): DBMS Integrity

Laboratory Session VII: BCNF Schema Analysis Tool

Writing a software module that reads data tables and business rules to check for normalization compliance. If a violation is found, the tool must output a lossless decomposition scheme that preserves data dependencies.

Laboratory Session VIII: Serializability Precedence Graph Check

Building a transaction monitoring component for database engines. The system reads concurrent transaction logs, builds directed dependency graphs, and blocks execution if a cycle is detected.

Laboratory Training Manual (Cont.): SQL Tuning Labs

Laboratory Session IX: Query execution Optimization Studies

Analyzing query plans generated by database engines for complex search queries. Students rewrite unoptimized search criteria to ensure the database can perform efficient index lookups instead of scanning entire datasets.

Laboratory Session X: High-Frequency Window Stream Deduplication

Building a financial processing system that filters incoming transactions in real-time. The solution uses a compact rolling queue to detect duplicate payments within short time windows, matching the results against an analytical database query plan.

Institutional Assessment Metrics & Grading Standards

Student evaluations follow a strict point-based system designed to measure both theoretical understanding and practical programming skills. Exams test architectural problem-solving capabilities under timed conditions.

Grading Breakdown Core Specification

Assessment Component	Weight Percentage	Passing Threshold
Midterm Theoretical Principles Examination	25%	75% Performance Mark
Final Integrated Engineering Assessment	35%	75% Performance Mark
Laboratory Performance Portfolios & Code Reviews	30%	80% Competency Proof
Peer-Reviewed Capstone Project Submissions	10%	100% Core Verification

Institutional Grading Standards (Cont.)

Theoretical Examinations Format

Exams require writing out formal proofs for algorithmic complexities and analyzing schema normalization levels manually. Students must trace execution steps, analyze memory layout changes, and identify race conditions in concurrent pseudo-code without relying on an interpreter.

Laboratory Assessment Guidelines

Code submissions are evaluated on computational correctness, style guidelines, and performance metrics. Solutions that meet accuracy requirements but use excessive memory or unoptimized search paths will face point deductions.

Senior Capstone Guidelines & Structural Milestones

The Senior Capstone is a year-long project where students apply their knowledge to solve a real-world computing problem. Projects must feature complex algorithmic structures and systematic database schema design.

Milestone 1: Architectural Proposal & Schema Review

Students submit a detailed project plan including data structure choices, class diagrams, and database schemas normalized to BCNF. The proposal must justify why specific data structures and index choices were selected for the target application workload.

Milestone 2: Prototype Validation & Telemetry Reports

The second milestone requires a working software prototype with integrated testing hooks. Students provide performance metrics confirming that memory allocations are tracked correctly and that database operations leverage appropriate index seek pathways.

Senior Capstone Guidelines (Cont.)

Milestone 3: Deployment Verification and Code Defense

The final phase requires deploying the complete system under realistic load conditions. Students present their work to a faculty panel, explaining their architectural decisions and defending their system design choices against potential edge-case failures.

Documentation Standards

Each project includes a technical report detailing the system architecture, mathematical optimization choices, database normalization steps, and concurrency management strategies. The report must match the style and detail of modern engineering specifications.

Appendix A: Academic Integrity Policy within Engineering Tracks

The department maintains a strict academic integrity policy. All code submissions must represent the independent work of the student or assigned team, and any outside references must be properly credited.

Plagiarism Definitions for Software Code

Plagiarism includes copying code segments from external sources without attribution, modifying variable names to obscure the original source, or using automated tools to generate solutions for individual assignments. All submissions are automatically screened using syntax-matching tools.

Disciplinary Action Procedures

Confirmed violations of the academic integrity policy result in an immediate zero grade for the assignment and referral to the institutional review board. Repeated offenses can lead to suspension or expulsion from the engineering degree program.

Appendix B: Laboratory Hardware Configuration Requirements

To run the laboratory exercises correctly, development workstations must meet specific hardware configurations capable of simulating multi-process and high-load database environments.

Workstation Hardware Minimum Specifications

- **Processor Unit:** Multi-core x86-64 CPU supporting hardware virtualizations.
- **Memory Array:** Minimum 16 Gigabytes of DDR4 RAM to support isolated virtual testing nodes.
- **Storage Layout:** High-speed solid-state drive with at least 50 Gigabytes allocated for database testing pools.

Software Engineering Tool Environments

Workstations are pre-configured with the standard CPython interpreter environment, profiling tools, and relational database testing servers. Code editors include syntax verification tools to enforce standard style conventions across all student submissions.

Appendix C: Industry Certification Framework Alignment

The curriculum is designed to align with major professional software engineering and database administration certification tracks, helping prepare students for professional career paths.

Professional Certification Mapping Matrix

Course Module	Target Certification Framework	Aligned Skill Competence
CS-102 OOP Frameworks	Professional Python Associate Standards	Advanced object construction and library design.
DATA-221 DBMS Core	Certified Relational Database Professional	Schema normalization and logical data modeling.
DATA-321 Advanced Transaction	Enterprise Database Tuning Specialist	Concurrency control and query optimization.

Continuing Professional Development Credit

Graduates can submit their completed course syllabus portfolios to professional engineering registries to verify continuing education credits, reducing the time required to earn advanced professional engineering designations.

Appendix D: Professional Reference Libraries & Digital Catalogs

Students have access to comprehensive digital research libraries and engineering databases to support their coursework and senior project research.

Institutional Research Database Access

The university library provides full access to online technical catalogs, computer science journals, and database systems conference archives. These resources allow students to reference historical papers and modern research to guide their system design choices.

Software Repository Infrastructure

The department hosts an internal version control server where students manage their source code, track project changes, and collaborate on team assignments. This repository infrastructure mirrors modern software development workflows, preparing students for professional environments.